

USING GRAPH NEURAL NETWORK TO SOLVE THE TRAVELING SALESMAN PROBLEM

Pingbang Hu, Jonathan Moore, Yi Zhou, Shubham Kumar Pandey, Anuraag Ramesh

University of Michigan



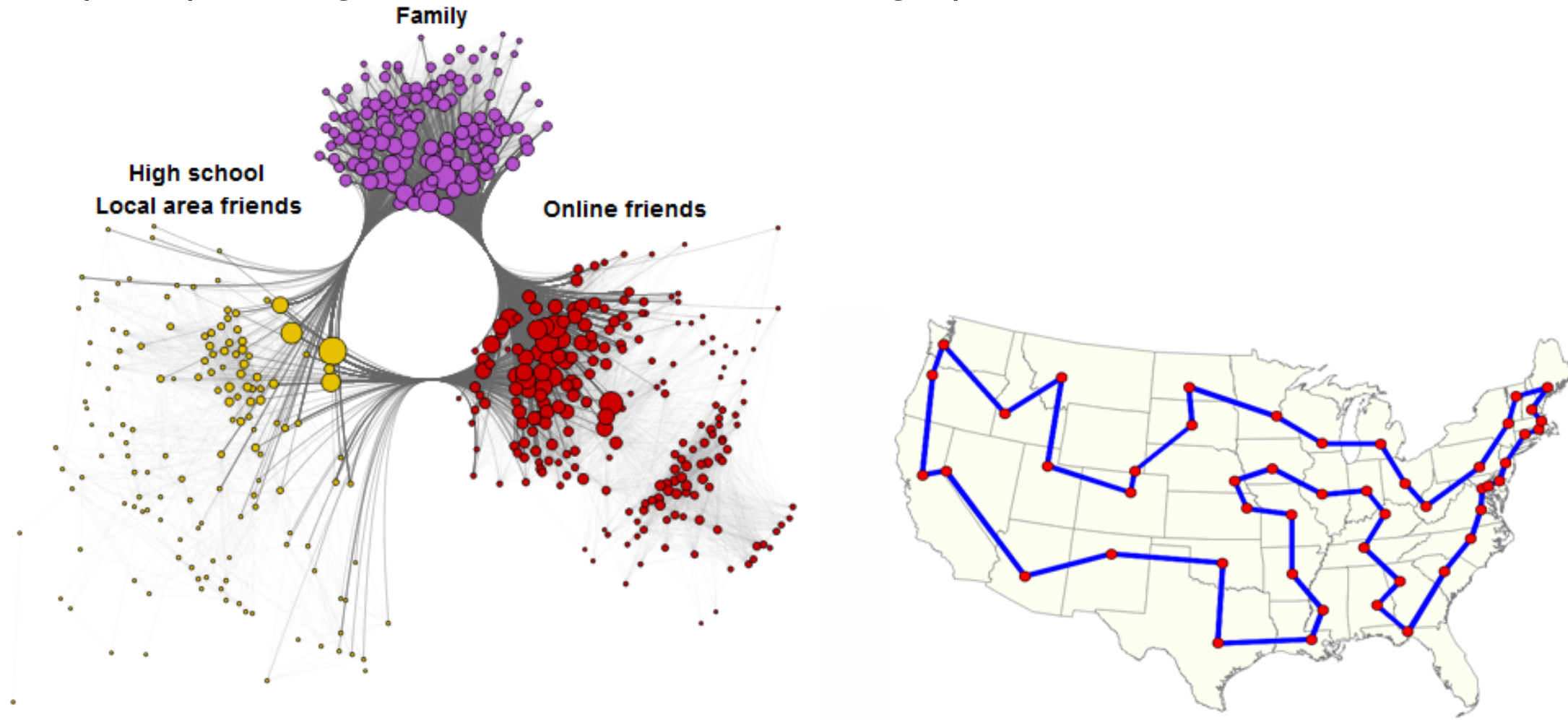
Abstract

Graph neural network is a rising concept in computer science that has demonstrated significant potential in many areas. Our project aims to further explore this potential by applying it to the classic traveling salesman problem. After generating the dataset, we first use imitation learning to obtain a pretty good path, which then will be inputted into reinforcement learning to obtain the optimal solution. Since imitation learning converges relatively rapidly, the method could be extremely fast compared to traditional exact solvers. Our next steps would be tuning the reinforcement learning, and running on larger TSP problems.

The Basics of GNN

A graph neural network (GNN) is a class of neural network for processing data best represented by graph data structures. They were popularized by their use in supervised learning on properties of various molecules.

Since their inception, variants of the message passing neural network (MPNN) framework have been proposed. These models optimize GNNs for use on larger graphs and apply them to domains such as social networks, citation networks, and online communities. GNNs have also been relatively successful in various NP-hard combinatorial problems, automated planning and path-planning areas due to the inherent graph structure of data.



Pipeline

1 Problem Formulation

1.1 TSP as Integer Linear Programming

We first formulate TSP in terms of **Integer Linear Programming**. Given an undirected weighted group $\mathcal{G} = (\mathcal{E}, \mathcal{V})$, we label the nodes with numbers $1, \dots, n$ and define

$$x_{ij} := \begin{cases} 1, & \text{if } (i, j) \in \mathcal{E}' \\ 0, & \text{if } (i, j) \in \mathcal{E} \setminus \mathcal{E}', \end{cases}$$

where $\mathcal{E}' \subset \mathcal{E}$ is a variable which can be viewed as a compact representation of all variables $x_{ij}, \forall i, j$. Furthermore, we denote the weight on edge (i, j) by c_{ij} , then for a particular TSP problem instance, we can formulate the problem as follows.

Pipeline (cont.)

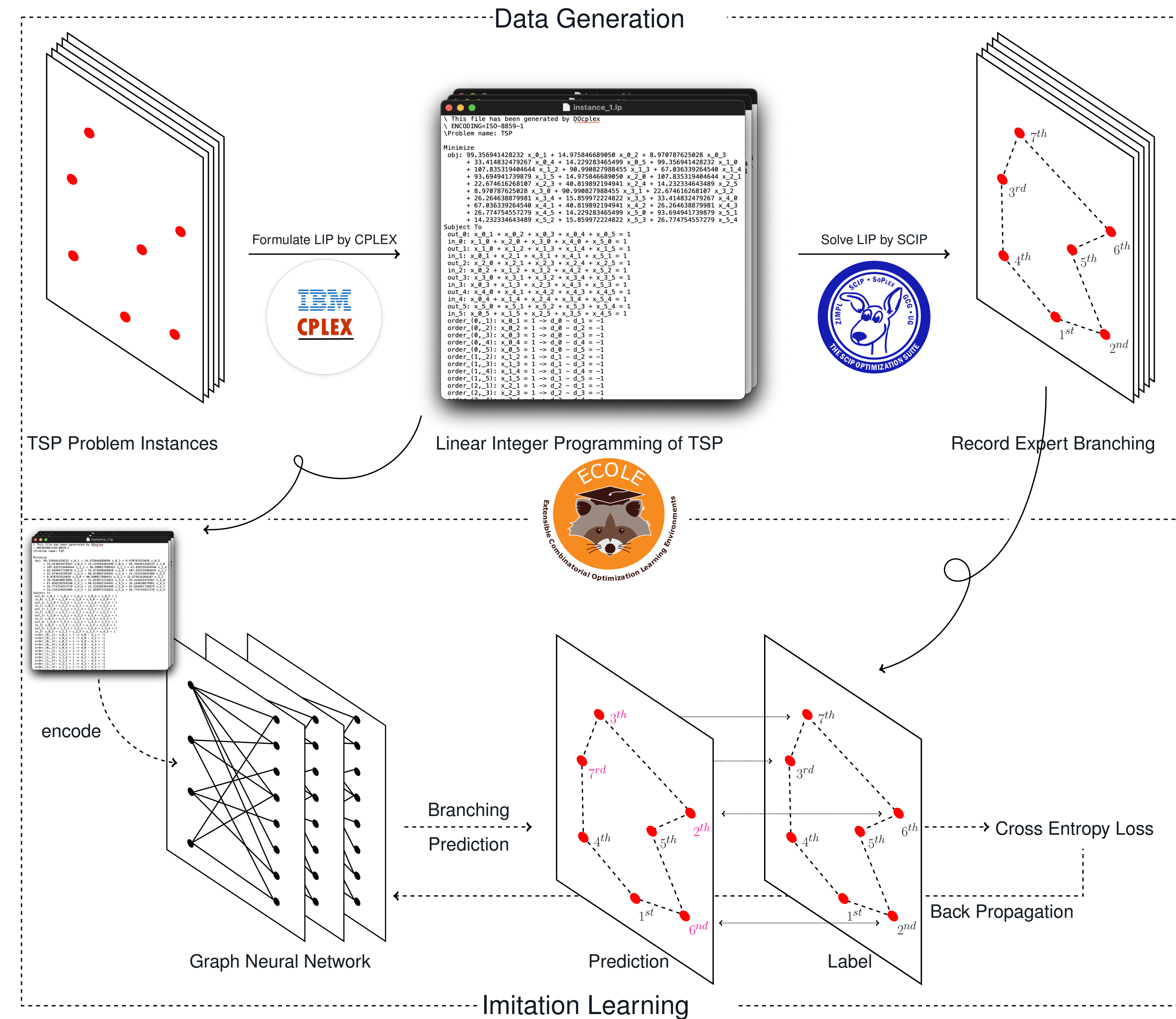
We can also divide the original relaxed LP into two sub-problems by **splitting the feasible region** according to a variable that does not respect integrality in the current relaxed LP solution x^* ,

$$x_i \leq \lfloor x_i^* \rfloor \vee x_i \geq \lceil x_i^* \rceil, \quad \exists i \leq p \mid x_i^* \notin \mathbb{Z}. \quad (1)$$

We see that by adding such additional constraint in two sub-problems respectively, we get a recursive algorithm called **Branch and Bound**[2].

2 Learning Pipeline

Our learning pipeline is as follows. Firstly, we only use imitation learning following [1] to obtain a good enough model, then we turn to unsupervised learning setup under the framework of reinforcement learning, which aims make the network learn some domain-specific property and help to boost the performance. The implementation summary can be found in



2.1 Imitation Learning

As mentioned before, the most effective branching strategy is strong branching, hence we train the network by behavioral cloning [**Efficient-Training-of-Artificial-Neural-Networks**].

We first run the SOTA solver and get state-action pairs:

$$\mathcal{D} = \{(s_i, \mathbf{a}_i^*)\}_{i=1}^N,$$

and then learn our policy $\tilde{\pi}^*$ by minimizing the cross-entropy loss:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum \log \tilde{\pi}_{\theta}(\mathbf{a}^* \mid \mathbf{s}).$$

Experimental Results

[TODO]

Conclusion

(Need to discuss together.) Even with a fairly limited scope and size of our neural network, we have already

Future Work

We will implement unsupervised reinforcement learning. After getting a stable model from imitation learning, we remove the expert's instructions and let the model explore by itself. Another way to further explore this topic is to perform on larger TSP datasets (e.g. TSP 50).

References